Cambridge O Level Computer Science

Chapter 1: Data Representation

Lecture notes with Check-Yourself questions and a full answer key.

In this chapter you will learn about:

- Number systems: why computers use binary; the denary, binary and hexadecimal systems; converting between them; uses of hexadecimal; binary addition and overflow; logical binary shifts; two's complement for negative numbers.
- Text, sound and images: character sets (ASCII and Unicode); how sound is sampled; how bitmap images are stored; sampling rate, sampling resolution, colour depth and image resolution.
- Data storage and compression: how storage is measured; calculating image and sound file sizes; why compression is needed; lossy and lossless compression.

1.1 Number systems

1.1.1 Why computers use binary

Every kind of data (numbers, text, sound, images, program instructions) must be converted into a binary format before a computer can process it. The reason is physical: a computer contains millions and millions of tiny electronic switches. Each switch can only be in one of two positions: ON or OFF.

- A switch in the ON position is represented by the binary digit 1.
- A switch in the OFF position is represented by the binary digit 0.
- Because there are only two possible states, the binary number system (which uses only 0 and 1) is the natural fit.
- These switches use logic gates to store and process data (logic gates are covered in Chapter 10).

So the basic building block of all data representation in a computer is the binary number system.

Check yourself

1. 1. Explain why a computer cannot work directly with denary (base 10) numbers.

sakib sir

2. 2. A tiny electronic switch inside a computer is in the ON position. Which binary digit represents this state?
3. 3. What is the connection between the binary digits 0 and 1 and the logic gates inside a computer?

1.1.2 The binary, denary and hexadecimal number systems

The denary system

Denary is the number system we use every day. It is a base 10 system: it uses ten digits (0 to 9) and each column heading is a power of 10.

Headings: 10,000 (10^4), 1,000 (10^3), 100 (10^2), 10 (10^1), 1 (10^0). For example, 25177 means 2x10,000 + 5x1,000 + 1x100 + 7x10 + 7x1.

The binary system

Binary is a base 2 system: it uses only the two digits 0 and 1, and each column heading is a power of 2: 2^0, 2^1, 2^2, 2^3, and so on.

The headings for an 8-bit binary number are: 128, 64, 32, 16, 8, 4, 2, 1. A typical binary number is 11101110. An 8-bit number uses 8 binary digits (bits).

Converting binary to denary

Method: write the column headings above the binary digits. Everywhere a 1 appears, add that column heading to a running total. Zeros contribute nothing.

Worked example: Convert 10110101 to denary.

Headings: 128 64 32 16 8 4 2 1

Digits: 1 0 1 1 0 1 0 1

Add the headings under the 1s: 128 + 32 + 16 + 4 + 1 = 181.

Answer: 10110101 in binary = 181 in denary.

Worked example: Convert 01101001 to denary.

Headings: 128 64 32 16 8 4 2 1

Digits: 0 1 1 0 1 0 0 1

sahib sir

Add: 64 + 32 + 8 + 1 = 105.

Answer: 105.

Check yourself

4. 1. Convert 11001010 to denary. Show your working.
5. 2. Convert 00011111 to denary.
6. 3. Convert 10000001 to denary.
7. 4. A 12-bit binary number is 000100101100. Convert it to denary. (Hint: the headings are 2048, 1024, 512, 256, 128, 64, 32, 16, 8, 4, 2, 1.)

Converting denary to binary

There are two methods. Both give the same answer.

Method 1: successive subtraction of powers of 2. Subtract the largest power of 2 that fits, write 1 under its heading, and repeat with the remainder until nothing is left. Put 0 under every heading you did not use.

Worked example: Convert 173 to 8-bit binary (Method 1).

173 - 128 = 45 -> 1 under 128

45 - 32 = 13 -> 1 under 32

13 - 8 = 5 -> 1 under 8

5 - 4 = 1 -> 1 under 4

1 - 1 = 0 -> 1 under 1

Headings used: 128, 32, 8, 4, 1.

Answer: 1 0 1 0 1 1 0 1.

Method 2: successive division by 2. Divide the number by 2, write down the remainder (0 or 1), and keep dividing the result by 2 until you reach 0. Then read the remainders from bottom to top. Add leading zeros if you need a fixed number of bits.

sakib sir

Worked example: Convert 173 to binary (Method 2).

173 / 2 = 86 remainder 1

86 / 2 = 43 remainder 0

43 / 2 = 21 remainder 1

21 / 2 = 10 remainder 1

10 / 2 = 5 remainder 0

5 / 2 = 2 remainder 1

2 / 2 = 1 remainder 0

1 / 2 = 0 remainder 1

Reading the remainders from bottom to top: 10101101. This matches Method 1.

Check yourself

8. 1. Convert 94 to 8-bit binary using Method 1 (subtraction).
9. 2. Convert 94 to binary using Method 2 (division). Do both methods agree?
10. 3. Convert 201 to 8-bit binary using either method.
11. 4. Convert 1000 to 16-bit binary. (The largest power of 2 below 1000 is 512.)

The hexadecimal system

Hexadecimal (hex) is a base 16 system. Because it needs 16 different digits, it uses the denary digits 0 to 9 plus the letters A to F:

- A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.

The column headings are powers of 16: ..., 65536 (16^4), 4096 (16^3), 256 (16^2), 16 (16^1), 1 (16^0). A typical hex number is 21F3A.

Hexadecimal is closely related to binary because 16 = 2^4. This means exactly FOUR binary digits are equivalent to ONE hexadecimal digit. The table below links the three systems:

Binary	Hexadecimal	Denary
0000	0	0

sakib sir